CyberOS — Complete Code & Architecture Documentation

AITAM College Hacksprint 2.0 — PS#5 Cyber Shield

1. Project Overview

CyberOS is a multimodal cybersecurity platform designed for the PS#5 Cyber Shield mandate. It analyzes URLs, emails, SMS, QR codes, and web content using localized detection engines and ML, translating diverse signals into actionable security evidence, correlated cases, explainable risk, and contextual education.

2. Problem Statement Mapping

PS#5 (Cyber Shield): CyberOS fulfills the core mandate by providing end-to-end detection, investigation, and reporting for Phishing, Scam, and Cyber-Fraud across all consumer attack vectors (SMS, QR, Web, Email). **PS26145 (Network Behavior):** CyberOS integrates passive network telemetry (Zeek) and ML-based behavioral anomaly detection (XGBoost) to evaluate post-click infrastructure and malware communications.

3. System Architecture

CyberOS operates via a decoupled microservices architecture:

`mermaid graph TD User([User Payload]) --> API[FastAPI Gateway] API --> Ext[Feature Extractors] API --> TI[Threat Intel Adapter] API --> DB[(MongoDB Cases)]

```
Net([Network Traffic]) --> Zeek[Zeek Sensor]
Zeek --> RP[Redpanda Kafka]
RP --> ML[XGBoost ML Worker]
ML --> DB

DB --> UI[Next.js Dashboard]
UI --> Analyst([SOC Analyst])
```

`

4. Technology Stack

- **Backend:** Python 3.11, FastAPI, Pydantic, Uvicorn, Playwright.
- **Frontend:** TypeScript, Next.js 14, React, Tailwind CSS.
- **Data & State:** MongoDB 7.0, Redis 7, Redpanda.
- **Machine Learning:** XGBoost, Scikit-learn.
- **Observability:** Prometheus, Grafana.

5. Module Architecture

- **backend/api/**: RESTful endpoints for case management, live scans, and network tunneling.
- **backend/content/**: Payload extraction (Regex, Entropy, Tokenization) for SMS, URLs, Emails, and QR codes.
- **backend/correlation/**: Entity graph engine mapping relationships.
- **backend/ml/**: Model registry and inference workers.

6. Detection Engines

```
mermaid pie title "Detection Engine Utilization by Modality" "Lexical & Heuristic" : 40 "XGBoost Tabular ML" : 35 "Threat Intelligence" : 15 "Web Sandboxing (DOM)" : 10
```

7. ML Architecture

- **Model Type:** XGBoost Classifier.
- **Features:** Tabular connection statistics (duration, orig_bytes, resp_bytes, state, history).
- **Inference:** Streaming worker consumes 5-second aggregated flow windows, predicting malicious network activity.

8. Dataset & Model Documentation

- **Training Corpus:** Synthesized network PCAPs and open-source phishing URLs.
- **Leakage Control:** Strict temporal and domain-based splits to prevent overlap between training and validation.

9. Threat Intelligence

External API adapters enrich local detection by checking domain reputation and IP blocklists. Designed to fail-open if upstream providers timeout.

10. Web Sandbox / Security

SSRF Protection Workflow: `mermaid sequenceDiagram participant API as FastAPI participant Sec as SSRF Filter participant Web as Target Website API->>Sec: Request URL Sec->>Sec: Resolve IP Sec->>Sec: Check against RFC-1918 (Private IP) alt is Private Sec-->>API: BLOCKED (Security Exception) else is Public Sec->>Web: Fetch DOM / Screenshot Web-->>API: Evidence Payload end`

11. Evidence Fabric

The CyberEvidence schema cryptographically tracks the provenance of every decision, mapping the exact model version and input hash.

12. Entity Graph

Translates flat alerts into a connected graph of EntityNodes (IP, URL, Email) and EntityEdges (resolves_to, contains).

13. Risk Engine

Normalizes ML confidence and heuristic weights into a 0-100 isk_score, translating technical feature importance into a 5-layer natural language explanation.

14. API Documentation

- **POST /api/scan**: Multi-modal payload ingestion.
- **GET /api/cases/{case_id}**: Retrieve full incident history.
- **GET /api/network/tunnels**: Aggregated uni-directional flows.

15. Database Architecture

MongoDB serves as the permanent system of record. Redis acts as a short-lived state store for network packet window aggregation.

16. Docker Deployment

Orchestrated via docker-compose.yml featuring 10 microservices isolated via internal Docker bridge networks.

17. Environment Configuration

Controlled via .env injection at runtime dictating log levels, API keys, and feature flags.

18. Testing & QA

E2E QA Script (scripts/torture/run_100.py) validates multi-modal payloads against the live API, persisting scoring logic.

19. Security Testing

Verified against payload injection, SSRF, XSS (in UI formatting), and denial-of-service via rate-limited processing queues.

20. Performance Testing

REST API returns content scans in <300ms. ML Worker handles thousands of network flows per second via Redpanda batching.

21. Resilience Testing

API gracefully degrades if MongoDB or Redpanda crash, ensuring content scanning functions independently of network observability.

22. Reporting

Automated JSON/PDF report generation summarizes identified IOCs, risk scores, and attack vectors for SOC analysts.

23. Education / Awareness

Every case generates contextual "Action" advice, educating the user on how the threat operated.

24. Known Limitations

- Browser analysis is computationally heavy.
- Multilingual SMS parsing is restricted to English heuristics.

25. Future Roadmap

- Kubernetes migration for ML worker auto-scaling.
- Deep Learning Transformer integration.
- Native SIEM/SOAR integrations.